

你的方向现在极其清晰了，而且你自己搭的8阶段框架非常对路。Android Display Stack 确实是手机底软里最硬核、最贯通的方向之一——从 App 的 OpenGL 调用到 OLED 的每个像素点亮，全链路都要懂。

我先给你一个总评：你这套框架不需要推翻，只需要在每个阶段里填入具体的"读什么、做什么、输出什么"。你底软出身（C/kernel/寄存器/时序强），薄弱区在 Android Framework 和 C++，所以前4阶段要下重手，后4阶段是你的舒适区。

下面是我帮你细化的 Step-by-Step 学习方案，每阶段包含：目标 → 核心资料（英文为主）→ 动手实践 → 输出物 → 时间建议。

全局主线（每天默念）

"一个红色像素，从用户点击 App 到 OLED 发红光，经历了哪些缓冲区、哪些进程、哪些驱动、哪些物理信号？"

所有学习都围绕这个问题展开。建议你把这张图贴在桌面：

```
App (Canvas/Skia/OpenGL)
  ↓ 画到 Surface (BufferQueue生产者)
SurfaceFlinger (system_server / 单独进程)
  ↓ 合成 (GPU 或 HWC)
HWComposer HAL (vendor实现)
  ↓ 提交到 DRM/KMS
DRM Driver (kernel)
  ↓ Plane → CRTC → Encoder → Connector
MIPI DSI Controller (SoC内部)
  ↓ LP/HS 模式, DCS命令
Panel Driver (OLED/LCD)
  ↓ 亮像素
你的眼球
```

Phase 1: Display First Principle（显示本质）

目标：理解"一帧图像是什么"，建立像素/刷新/缓冲的物理直觉。

时间：1.5周

必读资料

优先级 资料 说明

★★★★★ LearnOpenGL - Framebuffers ([链接](#)) 最好的入门，理解什么是 FBO、颜色附件、后处理

★★★★★ Android Graphics: Surface and SurfaceHolder ([AOSP](#)) 官方文档，先看 "Displays"、"Surface" 章节

★★★★★ 《Computer Graphics: Principles and Practice》(Foley) 第2章 Raster Graphics、第18章 Display Devices，只看这俩

★★★★★ Wikipedia: Double Buffering / VSync / Color Space 快速建立概念

动手实践

1. Linux fb0 实验：找台 Linux 设备（旧手机/树莓派/PC），直接写 `/dev/fb0`：

```
int fb = open("/dev/fb0", O_RDWR);
unsigned short *p = mmap(NULL, screensize, PROT_READ|PROT_WRITE,
MAP_SHARED, fb, 0);
// 画一条红色横线 (RGB565: 0xF800)
for(int x=100; x<300; x++) p[240*800 + x] = 0xF800;
```

目的：亲手触摸像素，理解 stride、bpp、framebuffer 是块裸内存。

2. VSync 体感实验：写一个程序交替清屏为红黑两色，先不加 `usleep`（撕裂），再加 `ioctl(FBIO_WAITFORVSYNC)`（不撕裂）。亲手感受 VSync 的意义。

输出物

- 一篇笔记：《Frame Buffer 不是帧，是一块裸内存》
- 手绘图：Double Buffer vs Triple Buffer 的时序图

Phase 2: Linux DRM/KMS（内核显示子系统）

目标：理解 kernel 里怎么管理显示。从 fbdev 进化到 DRM 的历史必须懂，否则看不懂代码。

时间：2.5周

必读资料

优先级 资料 说明

★★★★★ Kernel Doc: DRM/KMS Overview ([Documentation/gpu/drm-kms.rst](https://www.kernel.org/doc/html/latest/gpu/drm-kms.html)) 官方文档，必读

★★★★★ LWN: The DRM/KMS subsystem (Daniel Vetter) LWN 搜索 "Kernel mode setting"，系列文章约5篇，这是业界最好的 DRM 入门

★★★★★ Daniel Vetter's Blog (blog.ffwll.ch) DRM maintainer 的博客，搜 "drm beginners"

★★★★★ elinux.org: Display 嵌入式视角的总结

核心概念（必须画出来）

```
Framebuffer ( GEM Object / dma_buf )
    ↓ attach to
Plane (Primary/Overlay/Cursor)
    ↓ feed into
CRTC (Timing Controller, 负责分辨率/刷新率)
    ↓ feed into
Encoder (MIPI/LVDS/DP/eDP/HDMI)
    ↓ connect to
Connector (物理接口)
    ↓ drives
Panel (OLED/LCD)
```

动手实践

1. PC Linux 跑 modetest (安装 `libdrm-tests`):

```
modetest -M amdgpu    # 或 -M i915 / -M msm
modetest -s 32:1920x1080 -v # 点亮一个测试 pattern
```

目的：亲眼看到 DRM Pipeline 怎么工作，CRTC/Plane/Connector 的 ID 怎么对应。

2. 读一个最小 DRM Driver: `drivers/gpu/drm/tiny/` 下的 `bochs-drm.c` 或 `simplifiedrm.c`。这是最简单的 DRM 驱动，理解 `drm_driver` 结构体、`mode_config` 初始化。

3. 画一张你自己的 DRM 架构图：包含 GEM、KMS、fbdev 兼容层的关系。

输出物

- 笔记：《为什么 Linux 抛弃了 fbdev? DRM/KMS 第一性原理》
 - 手绘 DRM Pipeline 图（以后面试/工作天天用）
-

Phase 3: Android Graphics Stack (Android 显示框架)

目标：理解 Android 怎么在 DRM 之上搭积木。这是你最薄弱但最重要的阶段。

时间：3周（下重手）

必读资料

优先级 资料 说明

★★★★★★ AOSP: Graphics Architecture (source.android.com/docs/core/graphics) 官方总纲，逐页读

★★★★★★ AOSP: BufferQueue and Gralloc ([链接](#)) 理解 Buffer 生产消费模型

★★★★★★ AOSP: SurfaceFlinger and Hardware Composer ([链接](#)) 核心中的核心

★★★★★★ AOSP: Vulkan/Graphics Drivers 了解 GPU 驱动怎么接进来

★★★★★★ 《Android系统源代码情景分析》(罗升阳) 老但经典，看第12章 SurfaceFlinger、第11章 BufferQueue

核心概念（必须打通）

- Surface: App 的画布，本质是 BufferQueue 的 Producer
- BufferQueue: 生产者-消费者，GraphicBuffer 的队列
- GraphicBuffer: 带 fd 的缓冲区，底层是 Gralloc 分配的 dma-buf
- SurfaceFlinger: system_server 里的合成服务（或单独进程，看版本），负责把多个 Surface 合成一层
- HWComposer (HWC2): HAL 层，决定哪些层用 GPU 合成，哪些用硬件 Overlay Plane
- VSync: SF 和 App 的节拍器

动手实践

1. AOSP 代码硬读（下载源码，不用全编译）：

重点文件

frameworks/native/services/surfaceflinger/SurfaceFlinger.cpp

frameworks/native/libs/gui/BufferQueueProducer.cpp

frameworks/native/libs/gui/BufferQueueConsumer.cpp

hardware/interfaces/graphics/composer/2.1/IComposer.hal # HWC2 接口

目标：找到 `setLayerBuffer()`、`presentDisplay()` 的调用链。

2. adb dumpsys 实操：

```
adb shell dumpsys SurfaceFlinger --display-id      # 看显示设备
adb shell dumpsys SurfaceFlinger --latency         # 看 VSync 延迟
adb shell dumpsys SurfaceFlinger --visible         # 看可见图层
```

目的：建立"代码"和"现场"的对应关系。

3. systrace/Perfetto 看 SurfaceFlinger：

抓 trace，找到 `SurfaceFlinger` 线程，看它在 `onMessageReceived` 里做了什么。

输出物

- 笔记：《SurfaceFlinger 不是画图的，是拼图的》
- 一张大图：App → Surface → BufferQueue → SF → HWC → DRM 的完整数据流
- 用 C++ 写（或读懂）一个最小化的 `ANativeWindow` 使用示例

Phase 4: Buffer Management（缓冲区与零拷贝）

目标：理解 Android 为什么能做到零拷贝，这是显示性能的核心。

时间：1.5周

必读资料

优先级 资料 说明

★★★★★ Kernel Doc: dma-buf ([Documentation/driver-api/dma-buf.rst](https://www.kernel.org/doc/html/latest/driver-api/dma-buf.rst)) 官方，必读

★★★★★ AOSP: Gralloc HAL ([链接](#)) 理解 GraphicBuffer 怎么分配

★★★★★ LWN: DMA buffers 搜索 "dma_buf" 系列

★★★★★ Android Fence/Sync ([AOSP](#)) 理解 GPU 和 Display 的异步同步

核心概念

- ION / DMA-BUF Heaps：物理连续或非连续内存，跨设备共享
- Fence：GPU 渲染完没？Display 读完了没？防止读写竞争
- Cache Coherency：CPU/GPU/Display 三方看同一块内存，Cache 怎么同步
- IOMMU：把分散的物理页映射成连续的 IO 虚拟地址

动手实践

1. 读 Gralloc 实现：找 AOSP 或高通的 `gralloc` 实现，看 `alloc()` 怎么调 `ion_alloc()` 或 `dma_heap`。
2. 理解 Fence 流转：

```
adb shell cat /d/sync/info # 看当前系统的 fence 状态
```

在 systrace 里找 `fence` 信号线。

3. (可选) 写一个 dma-buf 共享 demo：两个进程通过 dma-buf fd 共享一块内存，一个写 RGB 数据，一个读。

输出物

- 笔记：《为什么 Camera 和 Display 可以共享同一块 Buffer? 》
- 手绘图：Fence 在 GPU → SF → HWC → DRM 之间的传递时序

Phase 5: Display Driver & Panel Bringup (显示驱动与点屏)

目标：进入你的舒适区——驱动。但要从“能点亮”进化到“理解每一行初始化代码”。

时间：2周

必读资料

优先级 资料 说明

★★★★★ Kernel: drivers/gpu/drm/panel/ 读 3 个 panel driver，比读 100 页文档有用

★★★★★ Kernel: DRM Panel Helper ([Documentation/gpu/drm-panel.rst](#)) 官方，理解 prepare/enable/disable/unprepare

★★★★★ Device Tree Bindings: [Documentation/devicetree/bindings/display/](#) 看懂 panel、bridge、dsi 的 DT 怎么写

★★★★★ 高通/MTK 内部文档（入职后获取） 各平台 DSI 控制器寄存器手册

核心概念

- Panel 生命周期： `prepare()` → `enable()` → `disable()` → `unprepare()`
- Backlight: PWM 或 WLED，通常单独一个 driver
- Bridge: DSI-to-LVDS 等转换芯片，有时在 panel 里，有时单独
- Regulator / GPIO / Reset / Clock: 点屏前必须上电、复位、给时钟
- DCS (Display Command Set): MIPI 定义的标准命令集

动手实践

1. 精读一个真实 Panel Driver:

推荐 `drivers/gpu/drm/panel/panel-samsung-s6e3hc2.c` 或 `panel-boe-xxx.c`。

逐行读: `probe()` 里怎么解析 DT? `prepare()` 里发哪些 DCS 命令? `enable()` 怎么开背光?

2. 画 Panel 初始化序列图:

```
Power On → Reset High → Delay 10ms → Send 0x11 (Sleep Out) → Delay 120ms → Send 0x29 (Display On) → Backlight On
```

目的: 以后黑屏时, 你知道断在哪一步。

3. 用逻辑分析仪/示波器看 MIPI (如果有条件): 看 LP-11 → LP-00 → HS 的切换, 看 DCS 命令的波形。

输出物

- 笔记: 《点屏不是点亮屏幕, 是说服屏幕听话》
- 一张 Panel Driver 状态机图

Phase 6: MIPI DSI (物理层与协议层)

目标: 理解 MIPI DSI 怎么传像素和命令, 这是黑屏/花屏/闪屏的根因区。

时间: 1.5周

必读资料

优先级 资料 说明

★★★★★ MIPI Alliance: DSI Specification (mipi.org) 官方, 注册后可下载, 看 Part 1 (协议)

★★★★★ NXP AN: MIPI DSI Application Note 搜 "NXP MIPI DSI AN", 硬件视角极好

★★★★★ Rockchip DRM Driver Doc RK 平台的 DSI 文档, 中文社区翻译多

★★★★★ Kernel: `drivers/gpu/drm/msm/dsi/` 或 `mediatek/dsi/` 读 DSI host controller driver

核心概念

- LP Mode (Low Power): 传命令, 慢, 省电
- HS Mode (High Speed): 传图像数据, 快
- DCS vs Generic: DCS 是标准命令, Generic 是厂商自定义

- Video Mode vs Command Mode: Video 像 VGA 实时刷, Command 像 MCU 带显存
- TE (Tearing Effect): Panel 的同步信号, 防止撕裂
- Lane / Clock / HS_PREP / HS_ZERO / CLK_POST: 时序参数

动手实践

1. 读 DSI Host Driver: 高通的 `msm_dsi.c` 或 MTK 的 `mtk_dsi.c`, 看怎么配置 `dsi->mode_flags`、`dsi->format`、`dsi->lanes`。
2. 理解时序参数: 找一个 panel 的 DT, 看懂 `hsync-len`、`hback-porch`、`hfront-porch`、`vsync-len` 等。
3. (入职后) 抓 DSI 错误日志:

```
adb shell dmesg | grep -i dsi
adb shell dmesg | grep -i mipi
```

输出物

- 笔记: 《MIPI DSI 不是接口, 是一份严格的时序合同》
- 一张图: LP → HS → 传数据 → LP 的状态切换

Phase 7: Boot Logo & UEFI (启动显示链路)

目标: 理解从按下电源键到 Android BootAnimation 的显示链路。

时间: 1周

必读资料

优先级 资料 说明

★★★★★ UEFI Spec: Graphics Output Protocol (GOP) 搜 "UEFI GOP specification", 理解 framebuffer 怎么在 UEFI 阶段暴露

★★★★★ Kernel: SimpleDRM / SimpleFB ([Documentation/gpu/drm-usage.rst](https://www.kernel.org/doc/html/latest/gpu/drm-usage.rst)) 理解 Kernel 怎么接管 UEFI 的 framebuffer

★★★★★ AOSP: Boot Animation 看 `bootable/bootanimation/`

★★★★★ Kernel Doc: Framebuffer Handoff 理解 `memblock` 保留内存给 logo

核心链路


```
BootROM → UEFI (Splash Logo via GOP) → Kernel (simplifiedrm/simplefb) → Init → SurfaceFlinger → BootAnimation
```

动手实践

1. 改 Boot Logo: 如果有旧 Android 设备, 尝试替换 `logo.bin` 或修改 kernel 里的 `logo_linux_clut224.ppm`。
2. 理解 Kernel 怎么找 UEFI FB: 读 `drivers/firmware/efi/efi-init.c` 和 `drivers/gpu/drm/tiny/simplifiedrm.c`。

输出物

- 笔记: 《从 UEFI 到 BootAnimation: 启动阶段的显示接力》

Phase 8: Debug Methodology (分层定位法)

目标: 建立系统化的 Debug 思维, 这是你以后吃饭的本事。

时间: 贯穿始终, 最后集中总结

核心方法: 分层排除法

遇到黑屏/花屏, 绝不猜, 按层查:

层级 查什么 命令/工具

App 有没有提交 Buffer? `adb shell dumpsys SurfaceFlinger`

Framework SF 有没有合成? VSync 正常吗? `systrace/Perfetto`

HAL HWC 有没有收到 layer? `adb shell dumpsys hwcomposer`

Kernel DRM CRTC 有没有 commit? Plane 有没有数据? `modetest / dmesg \ | grep drm`

DSI DSI 有没有发数据? LP/HS 切换正常吗? `dmesg \ | grep dsi` / 示波器

Panel 初始化序列走完了吗? 背光开了吗? 读 `panel driver log` / 量 GPIO

必读资料

- AOSP: Debugging Display Issues (官方零散文档)
- 高通/MTK 内部 Troubleshooting Guide (入职后获取)

输出物

- 一张《Display Debug Cheat Sheet》(打印放桌上)
-

整体时间线建议

阶段 内容 时间 累计

1 Display First Principle 1.5周 1.5周

2 Linux DRM/KMS 2.5周 4周

3 Android Graphics 3周 7周

4 Buffer Management 1.5周 8.5周

5 Display Driver 2周 10.5周

6 MIPI DSI 1.5周 12周

7 Boot Logo 1周 13周

8 Debug + 复盘 持续 3个月+

建议：前8周每天投入2-3小时，后5周每天1-2小时。入职前完成 Phase 1-4，入职后结合业务深入 5-8。

给你的关键建议

1. 输出驱动输入

你之前写 SoC/DSP 系列的习惯非常好。建议直接开写《Android Display First Principle》系列博客，每阶段一篇。写的时候你会发现"以为自己懂了"和"真能讲清楚"之间的鸿沟。

2. 中英文资料配比

- 英文优先：Kernel Doc、AOSP Doc、LWN、MIPI Spec、Daniel Vetter Blog 都是第一手资料。
- 中文辅助：罗升阳的书、Rockchip/全志社区文档、CSDN 上少数高质量分析（注意甄别）。
- 绝不看：国内培训机构出的"Android Framework 面试题"类文章，对底软没用。

3. 你的优势区 vs 危险区

你的优势（要放大） 你的危险区（要警惕）

Kernel 驱动、寄存器、时序、C 语言 Android Framework C++ 代码阅读速度

调试死机、抓 log、硬件问题定位 SurfaceFlinger/HWC 的复杂状态机

MIPI/DSI/Panel 物理层 BufferQueue 的生产者-消费者并发逻辑

4. 立即开始的3件事（今晚就能做）

5. 读 LearnOpenGL Framebuffers 章节（1小时）

6. 在 Linux 上写 `/dev/fb0` 画线程序（2小时）

7. 下载 AOSP 源码（`repo init -u`

`https://android.googlesource.com/platform/manifest -b android-14.0.0_r1`

--depth=1，挂一晚上同步)

总结：你的8阶段框架是对的，方向是对的，现在只需要把每个阶段的具体资料和实践填进去，然后按时间表推进。Android Display 是一个"懂了之后非常通透"的子系统，因为它的数据流是单向的（App → Screen），比 Camera（双向控制+数据）和 Audio（实时性+复杂路由）都更容易建立第一性原理理解。

加油，这个方向选得很好，显示驱动工程师在手机厂商里一直是稀缺资源。